The last thing I'd like to talk about in this chapter is how to use DI with your filters. You saw in chapters 8 and 9 that DI is integral to ASP.NET Core, and in the next section you'll see how to design your filters so that the framework can inject service dependencies into them for you.

22.3 Using dependency injection with filter attributes

In this section you'll learn how to inject services into your filters so you can take advantage of the simplicity of DI in your filters. You'll learn to use two helper filters to achieve this, `TypeFilterAttribute` and `ServiceFilterAttribute`, and you'll see how they can be used to simplify the action filter you defined in section 22.1.3.

The filters we've created so far have been created as attributes. This is useful for applying filters to action methods and controllers, but it means you can't use DI to inject services into the constructor. C# attributes don't let you pass dependencies into their constructors (other than constant values), and they're created as singletons, so there's only a single instance of an attribute for the lifetime of your app. So what happens if you need to access a transient or scoped service from inside the singleton attribute?

Listing 22.6 showed one way of doing this, using a pseudo-service locator pattern to reach into the DI container and pluck out `RecipeService` at runtime. This works but is generally frowned upon as a pattern in favor of proper DI. So how can you add DI to your filters?

The key is to split the filter in two. Instead of creating a class that's both an attribute and a filter, create a filter class that contains the functionality and an attribute that tells the framework when and where to use the filter.